

Problem Set 8

BM2043 - Algorithms and Data Structure Lab

November 1, 2024

1D DP

Problem 1: Climbing Stairs

Problem Statement:

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example: Example 1:

- **Input:** $n = 2$
- **Output:** 2
- **Explanation:** There are two ways to climb to the top.
 1. 1 step + 1 step
 2. 2 steps

Example 2:

- **Input:** $n = 3$
- **Output:** 3
- **Explanation:** There are three ways to climb to the top.
 1. 1 step + 1 step + 1 step
 2. 1 step + 2 steps
 3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

Class Definition:

```
class Solution {
public:
    int climbStairs(int n) {
        // Your code goes here
    }
};
```

Problem 2: Frog Jump

Problem Statement:

You are given a **0-indexed integer array** 'stones' sorted in strictly increasing order representing the positions of stones in a river. A frog, initially on the first stone, wants to travel to the last stone and then return to the first stone. However, it can jump to any stone at most once.

The length of a jump is the absolute difference between the position of the stone the frog is currently on and the position of the stone to which the frog jumps.

More formally, if the frog is at 'stones[i]' and is jumping to 'stones[j]', the length of the jump is $|\text{stones}[i] - \text{stones}[j]|$. The cost of a path is the maximum length of a jump among all jumps in the path.

Your task is to return the minimum cost of a path for the frog.

Examples: Example 1:

- **Input:** stones = [0, 2, 5, 6, 7]
- **Output:** 5
- **Explanation:** The frog can take the path:
 1. Jump from stone '0' to '2' (length = 2)
 2. Jump from stone '2' to '5' (length = 3)
 3. Jump from stone '5' to '6' (length = 1)
 4. Jump from stone '6' to '7' (length = 1)
 5. Return to stone '0' (length = 7)

The maximum jump length in this path is '5', which is the minimum cost of the path.

Example 2:

- **Input:** stones = [0, 3, 9]
- **Output:** 9
- **Explanation:** The frog can jump directly to the last stone and return to the first stone. In this case, the length of each jump will be '9'. The cost for the path will be $\max(9, 9) = '9'$.

This is the minimum achievable cost.

Constraints:

- $2 \leq \text{stones.length} \leq 10^5$
- $0 \leq \text{stones}[i] \leq 10^9$
- $\text{stones}[0] == 0$
- 'stones' is sorted in a strictly increasing order.

Class Definition:

```
class Solution {
public:
    int maxJump(vector<int>& stones) {
        // Your code goes here
    }
};
```

Problem 3: Non-Adjacent Sum Problem

Problem Statement:

Given an integer array 'nums' of size n , return the maximum sum possible using the elements of 'nums' such that no two elements taken are adjacent in 'nums'.

A subsequence of an array is a sequence that can be derived from the array by deleting some or no elements without changing the order of the remaining elements.

Examples: Example 1:

- **Input:**

nums = [1, 2, 4]

- **Output:**

5

Explanation: The elements [1, 2, 4] are taken to get the maximum sum.

Example 2:

- **Input:**

nums = [2, 1, 4, 9]

- **Output:**

11

Explanation: The elements [2, 1, 4, 9] are taken to get the maximum sum.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^5$
- $0 \leq \text{nums}[i] \leq 1000$

Note: Can we get a solution with $O(n)$ time and $O(1)$ space complexity?

Class Definition:

```
class Solution {
public:
    int nonAdjacent(vector<int>& nums) {
        // Your code goes here
    }
};
```

Problem 4: House Robber Problem

Problem Statement:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, but adjacent houses have security systems connected that will automatically contact the police if two adjacent houses are robbed on the same night.

Given an integer array 'nums' representing the amount of money in each house, return the maximum amount of money you can rob tonight without alerting the police.

Examples: Example 1:

- **Input:**

```
nums = [1, 2, 3, 1]
```

- **Output:**

4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3). Total amount you can rob = 1 + 3 = 4.

Example 2:

- **Input:**

```
nums = [2, 7, 9, 3, 1]
```

- **Output:**

12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9), and rob house 5 (money = 1). Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

Class Definition:

```
class Solution {
public:
    int rob(vector<int>& nums) {
        // Your code goes here
    }
};
```

2D DP (Ninja's Training Problem)

Problem Statement:

A ninja has planned an n -day training schedule. Each day, he must perform one of three activities: running, stealth training, or fighting practice. The same activity cannot be done on two consecutive days, and the ninja earns specific merit points based on the activity and the given day.

Given a $n \times 3$ matrix, where:

- `matrix[i][0]` represents the merit points for running on day $i + 1$,
- `matrix[i][1]` represents the merit points for stealth training on day $i + 1$,
- `matrix[i][2]` represents the merit points for fighting practice on day $i + 1$,

Return the maximum possible merit points that the ninja can earn.

Examples: Example 1:

- **Input:** `matrix = [[10, 40, 70], [20, 50, 80], [30, 60, 90]]`
- **Output:** 210
- **Explanation:**
 - Day 1: Fighting practice = 70
 - Day 2: Stealth training = 50
 - Day 3: Fighting practice = 90
 - Total = $70 + 50 + 90 = 210$ (optimal points).

Example 2:

- **Input:** `matrix = [[70, 40, 10], [180, 20, 5], [200, 60, 30]]`
- **Output:** 290
- **Explanation:**
 - Day 1: Running = 70
 - Day 2: Stealth training = 20
 - Day 3: Running = 200
 - Total = $70 + 20 + 200 = 290$ (optimal points).

Constraints:

- $1 \leq n \leq 10^4$
- n == number of rows in the matrix
- 3 == number of columns in the matrix
- $0 \leq \text{matrix}[i][j] \leq 1000$

Class Definition:

```
class Solution {
public:
    int ninjaTraining(vector<vector<int>>& matrix) {
        // Your code goes here
    }
};
```

DP on Grids

Problem 1: Unique Paths with Obstacles Problem

Problem Statement:

You are given an $m \times n$ integer array `grid`. There is a robot initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in the grid. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

Examples: Example 1:

- **Input:** `obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]`
- **Output:** 2
- **Explanation:** There is one obstacle in the middle of the 3x3 grid above. There are two ways to reach the bottom-right corner:
 1. Right → Right → Down → Down
 2. Down → Down → Right → Right

Example 2:

- **Input:** `obstacleGrid = [[0,1],[0,0]]`
- **Output:** 1

Constraints:

- $m == \text{obstacleGrid.length}$
- $n == \text{obstacleGrid}[i].\text{length}$
- $1 \leq m, n \leq 100$
- `obstacleGrid[i][j]` is either 0 or 1.

Class Definition:

```
class Solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
        // Your code goes here
    }
};
```

Problem 2: Minimum Path Sum Problem

Problem Statement:

Given an $m \times n$ grid filled with non-negative numbers, find a path from the top-left corner to the bottom-right corner that minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Examples: Example 1:

- **Input:**

```
grid = [[1, 3, 1], [1, 5, 1], [4, 2, 1]]
```

- **Output:**

7

Explanation: The path $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$ minimizes the sum.

Example 2:

- **Input:**

```
grid = [[1, 2, 3], [4, 5, 6]]
```

- **Output:**

12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{grid}[i][j] \leq 200$

Class Definition:

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        // Your code goes here
    }
};
```

Problem 3: Triangle Problem

Problem Statement:

Given a triangle array, return the minimum path sum from top to bottom.

For each step, you may move to an adjacent number in the row below. More formally, if you are on index i in the current row, you may move to either index i or index $i + 1$ in the next row.

Examples: Example 1:

- **Input:**

```
triangle = [[2], [3, 4], [6, 5, 7], [4, 1, 8, 3]]
```

- **Output:**

11

Explanation: The triangle looks like:

```
  2
 3 4
6 5 7
4 1 8 3
```

The minimum path sum from top to bottom is $2 + 3 + 5 + 1 = 11$.

Example 2:

- **Input:**

```
triangle = [[-10]]
```

- **Output:**

-10

Constraints:

- $1 \leq \text{triangle.length} \leq 200$
- $\text{triangle}[0].\text{length} == 1$
- $\text{triangle}[i].\text{length} == \text{triangle}[i - 1].\text{length} + 1$
- $-10^4 \leq \text{triangle}[i][j] \leq 10^4$

Follow Up: Could you do this using only $O(n)$ extra space, where n is the total number of rows in the triangle?

Class Definition:

```
class Solution {
public:
    int minimumTotal(vector<vector<int>>& triangle) {
        // Your code goes here
    }
};
```

DP on Stocks

Problem 1: Best Time to Buy and Sell Stock

Problem Statement:

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Examples: Example 1:

- **Input:**

```
prices = [7, 1, 5, 3, 6, 4]
```

- **Output:**

5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = $6 - 1 = 5$. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

- **Input:**

```
prices = [7, 6, 4, 3, 1]
```

- **Output:**

0

Explanation: In this case, no transactions are done and the maximum profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^4$

Class Definition:

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // Your code goes here
    }
};
```

Problem 2: Best Time to Buy and Sell Stock II

Problem Statement:

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i th day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can buy it then immediately sell it on the same day.

Find and return the maximum profit you can achieve.

Examples: Example 1:

- **Input:** `prices = [7, 1, 5, 3, 6, 4]`
- **Output:** 7
- **Explanation:**
 - Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = $5 - 1 = 4$.
 - Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = $6 - 3 = 3$.
 - Total profit is $4 + 3 = 7$.

Example 2:

- **Input:** `prices = [1, 2, 3, 4, 5]`
- **Output:** 4
- **Explanation:** Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = $5 - 1 = 4$. Total profit is 4.

Example 3:

- **Input:** `prices = [7, 6, 4, 3, 1]`
- **Output:** 0
- **Explanation:** There is no way to make a positive profit, so we never buy the stock, achieving a maximum profit of 0.

Constraints:

- $1 \leq \text{prices.length} \leq 3 \times 10^4$
- $0 \leq \text{prices}[i] \leq 10^4$

Class Definition:

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // Your code goes here
    }
};
```

Problem 3: Best Time to Buy and Sell Stock III

Problem Statement:

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Examples: Example 1:

- **Input:** `prices = [3, 3, 5, 0, 0, 3, 1, 4]`
- **Output:** 6
- **Explanation:**
 - Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = $3 - 0 = 3$.
 - Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = $4 - 1 = 3$.
 - Total profit = $3 + 3 = 6$.

Example 2:

- **Input:** `prices = [1, 2, 3, 4, 5]`
- **Output:** 4
- **Explanation:** Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = $5 - 1 = 4$. Total profit = 4.

Example 3:

- **Input:** `prices = [7, 6, 4, 3, 1]`
- **Output:** 0
- **Explanation:** In this case, no transaction is done, achieving a maximum profit of 0.

Constraints:

- $1 \leq \text{prices.length} \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^5$

Class Definition:

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // Your code goes here
    }
};
```

Problem 4: Best Time to Buy and Sell Stock IV

Problem Statement:

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i -th day, and an integer k .

Find the maximum profit you can achieve. You may complete at most k transactions (i.e., you may buy at most k times and sell at most k times).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Examples: Example 1:

- **Input:**

`k = 2, prices = [2, 4, 1]`

- **Output:**

2

Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = $4 - 2 = 2$.

Example 2:

- **Input:**

`k = 2, prices = [3, 2, 6, 5, 0, 3]`

- **Output:**

7

Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = $6 - 2 = 4$. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = $3 - 0 = 3$. Total profit = $4 + 3 = 7$.

Constraints:

- $1 \leq k \leq 100$
- $1 \leq \text{prices.length} \leq 1000$
- $0 \leq \text{prices}[i] \leq 1000$

Class Definition:

```
class Solution {
public:
    int maxProfit(int k, vector<int>& prices) {
        // Your code goes here
    }
};
```

Problem 5: Best Time to Buy and Sell Stock with Transaction Fee

Problem Statement:

You are given an array `prices` where `prices[i]` is the price of a given stock on the i -th day, and an integer `fee` representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

Note:

- You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
- The transaction fee is only charged once for each stock purchase and sale.

Examples: Example 1:

- **Input:** `prices = [1,3,2,8,4,9]`, `fee = 2`
- **Output:** 8
- **Explanation:**
 - Buy at `prices[0] = 1`
 - Sell at `prices[3] = 8`
 - Buy at `prices[4] = 4`
 - Sell at `prices[5] = 9`
 - The total profit is $((8 - 1) - 2) + ((9 - 4) - 2) = 8$.

Example 2:

- **Input:** `prices = [1,3,7,5,10,3]`, `fee = 3`
- **Output:** 6
- **Explanation:** The optimal transactions yield a profit of 6.

Constraints:

- $1 \leq \text{prices.length} \leq 50,000$
- $1 \leq \text{prices}[i] < 50,000$
- $0 \leq \text{fee} < 50,000$

Class Definition:

```
class Solution {
public:
    int maxProfit(vector<int>& prices, int fee) {
        // Your code goes here
    }
};
```